웹 개발자 부트캠프 과정

CodingOn

Spring Boot 프로젝트 생성하기

Spring Boot 프로젝트 시작하기

https://start.spring.io/

- Spring Boot 기반으로 Spring 관련 프로젝트를 만들어주는 사이트
- Spring에서 운영하고 있으며, 해당 사이트에서 원하는 라이브러리를 선택 후 프로젝트 생성 가능

Spring Boot 프로젝트 시작하기

spring initializr

Project
- ● Gradle - Groovy
- ○ Gradle - Kotlin
- ○ Maven

Language
- ● Java
- ○ Kotlin
- ○ Groovy

Spring Boot
- ○ 3.3.2 (SNAPSHOT)
- ● 3.3.1
- ○ 3.2.8 (SNAPSHOT)
- ○ 3.2.7

Project Metadata

Group	codingon
Artifact	spring-boot-default
Name	spring-boot-default
Description	Demo project for Spring Boot
Package name	codingon.spring-boot-default
Packaging	● Jar ○ War
Java	○ 22 ○ 21 ● 17

Dependencies

ADD DEPENDENCIES... CTRL + B

No dependency selected

spring initializr

생성할 프로젝트의 빌드 자동화 툴

생성할 프로젝트의 언어

Project

- Gradle - Groovy
- Gradle - Kotlin
- Maven

Language

- Java
- Kotlin
- Groovy

스프링 부트 버전

Spring Boot

- 3.3.2 (SNAPSHOT)
- 3.3.1
- 3.2.8 (SNAPSHOT)
- 3.2.7

프로젝트 정보

Project Metadata

Group	codingon
Artifact	spring-boot-default
Name	spring-boot-default
Description	Demo project for Spring Boot
Package name	codingon.spring-boot-default
Packaging	Jar / War
Java	22 / 21 / 17

프로젝트 의존성

여기에 추가한 디펜던시가 **build.gradle** 파일에 추가됨!

Dependencies ADD DEPENDENCIES... CTRL + B

Spring Web WEB 스프링 기반으로 웹 개발

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Thymeleaf TEMPLATE ENGINES 템플릿 엔진

A modern server-side Java template engine for both web and standalone environments. Allows HTML to be correctly displayed in browsers and as static prototypes.

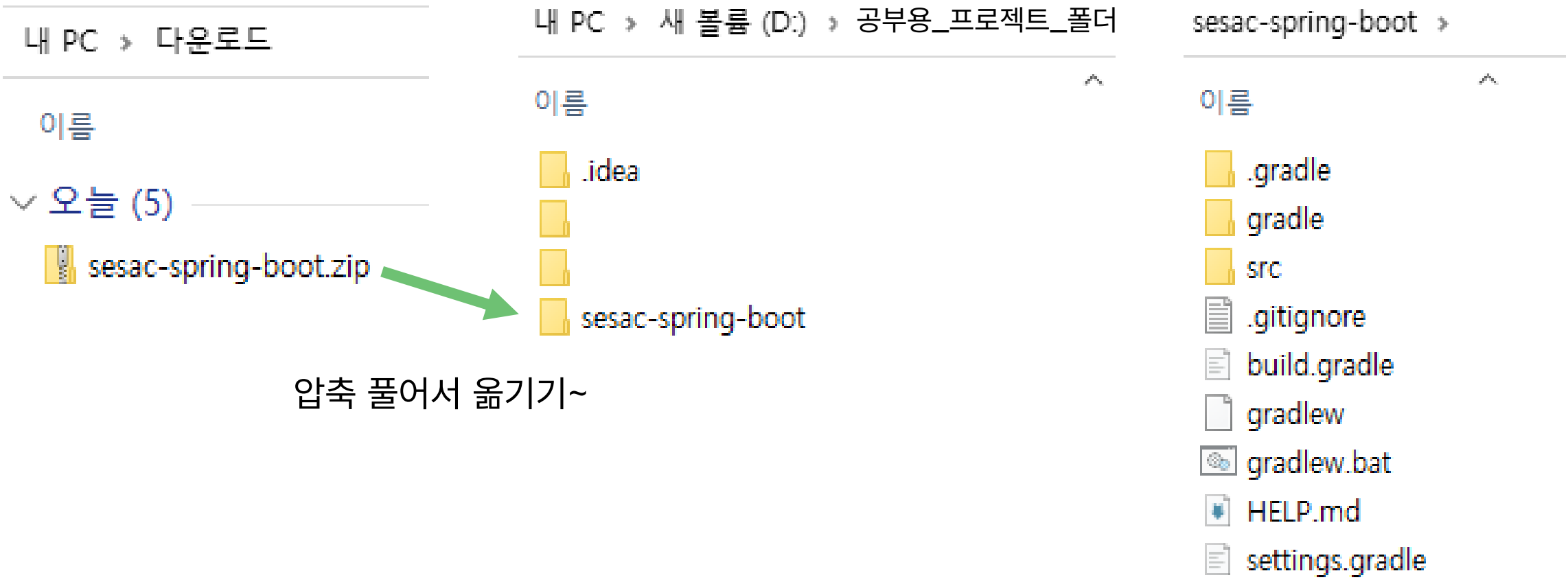
Lombok DEVELOPER TOOLS

Java annotation library which helps to reduce boilerplate code.

코드 다이어트 라이브러리 ex. getter, setter 등 코드를 간단히 사용하게 함

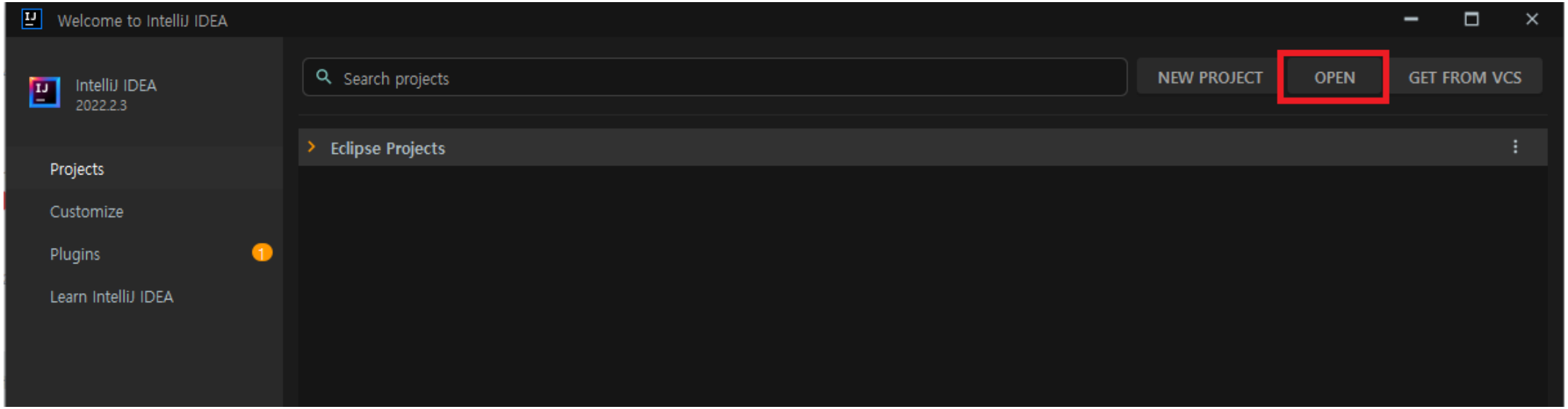
GENERATE CTRL + ⏎ | EXPLORE CTRL + SPACE | SHARE...

프로젝트 압축 파일 프로젝트 경로로 옮기기

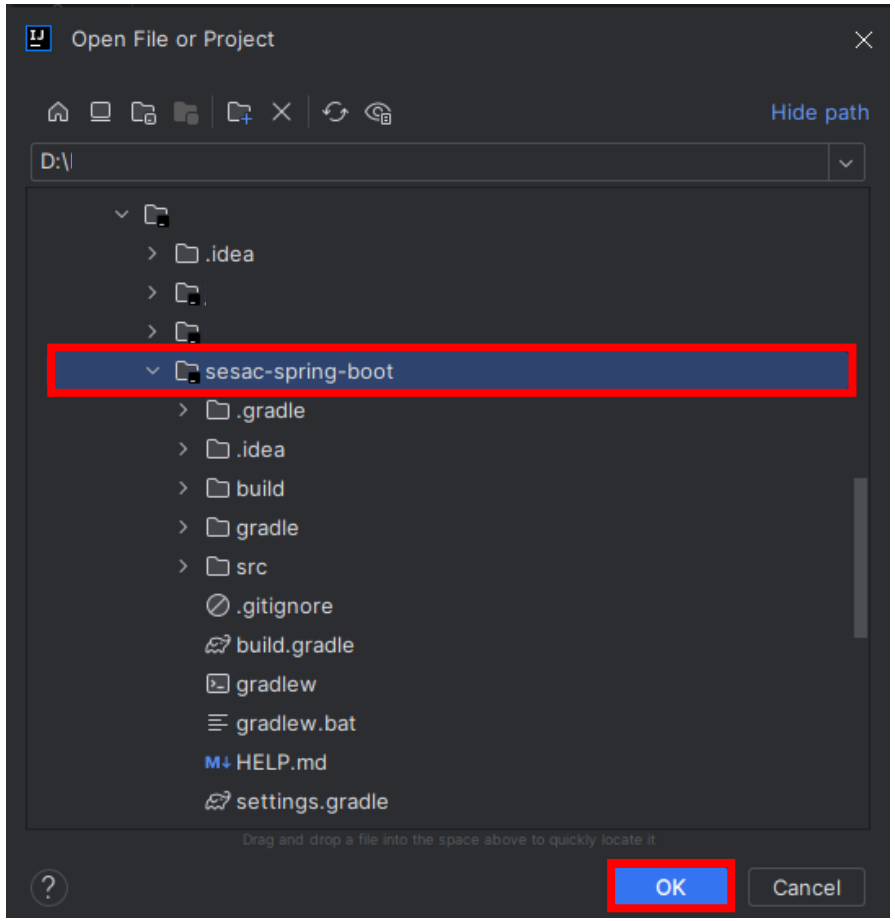


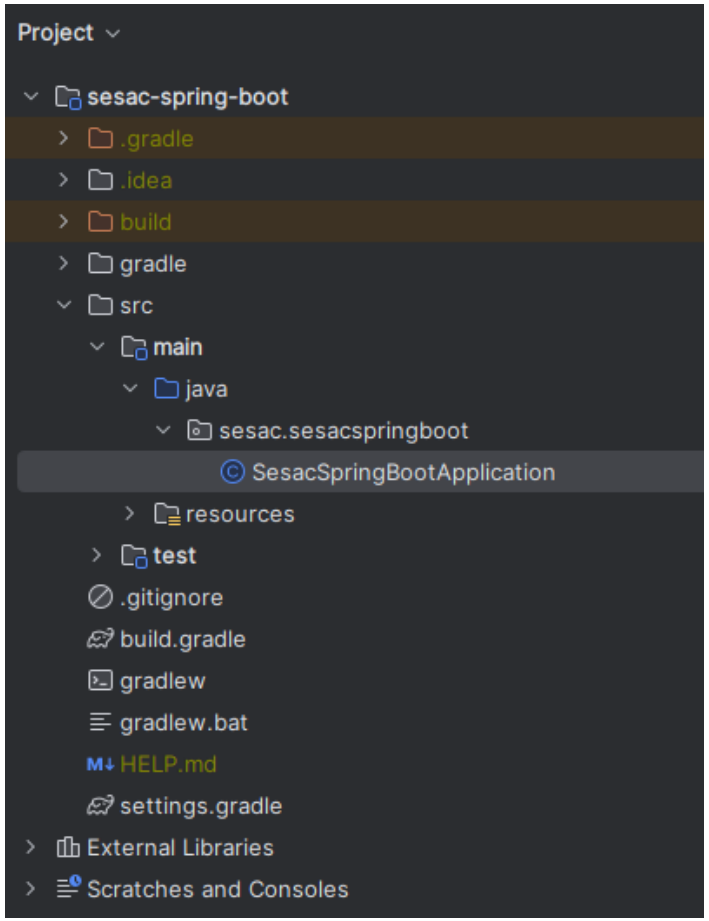
압축 풀어서 옮기기~

IntelliJ 에서 프로젝트 열기



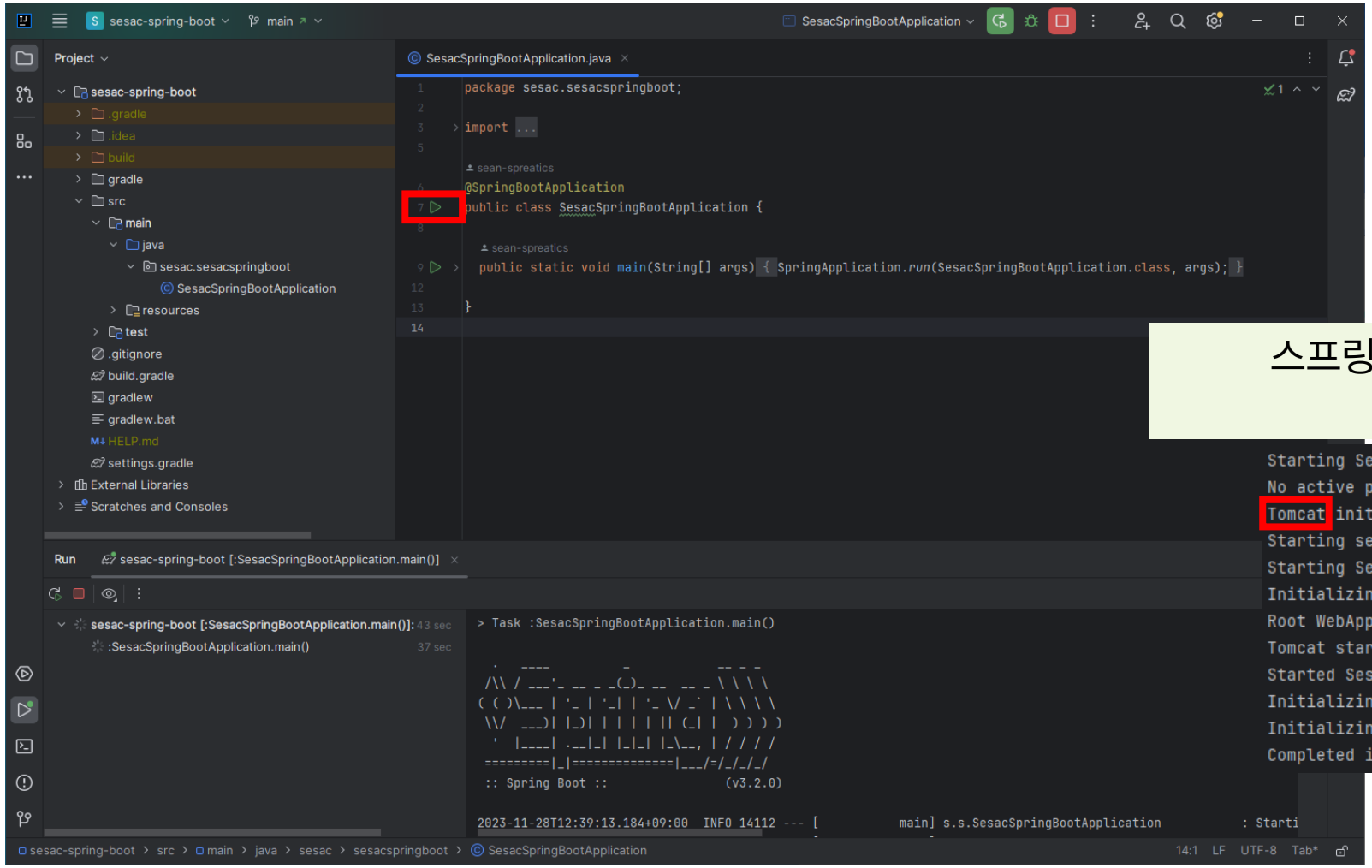
IntelliJ 에서 프로젝트 열기

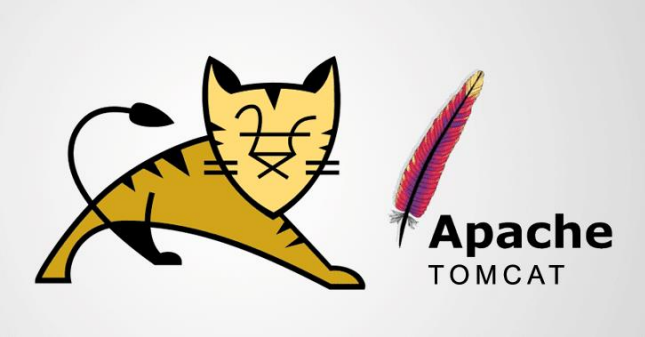




주의. src/ 폴더가 프로젝트이름 바로 아래에 위치하도록 OPEN 하기!

프로젝트 실행하기



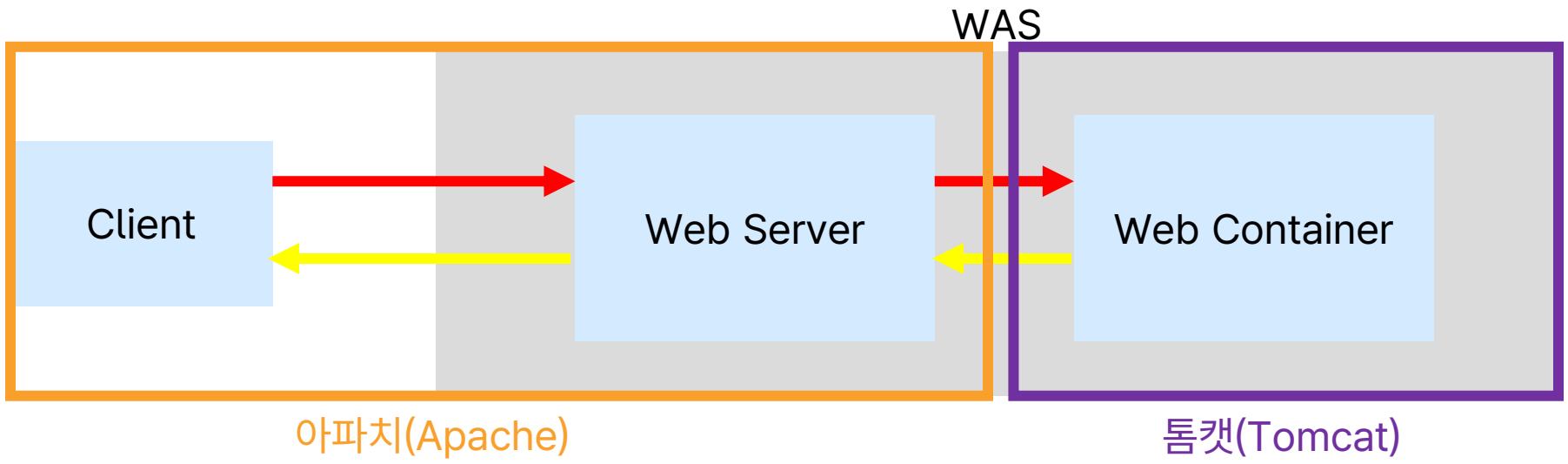


스프링 부트는 기본적으로 **8080 포트**를 사용하여
내장된 **Apache Tomcat 웹 서버**를 실행

```
Starting SesacSpringBootApplication using Java 17.0.8 with PID 14112
No active profile set, falling back to 1 default profile: "default"
Tomcat initialized with port 8080 (http)
Starting service [Tomcat]
Starting Servlet engine: [Apache Tomcat/10.1.16]
Initializing Spring embedded WebApplicationContext
Root WebApplicationContext: initialization completed in 857 ms
Tomcat started on port 8080 (http) with context path ''
Started SesacSpringBootApplication in 1.502 seconds (process running for 1.769)
Initializing Spring DispatcherServlet 'dispatcherServlet'
Initializing Servlet 'dispatcherServlet'
Completed initialization in 0 ms
```

잠깐! Apache Tomcat 이란?

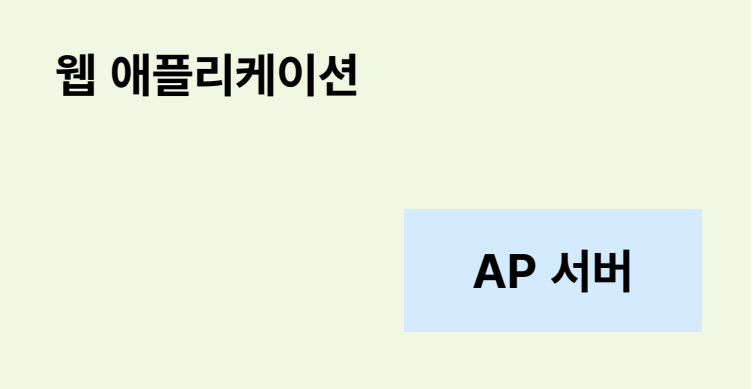
- 아파치 소프트웨어 재단에서 개발한 서블릿 컨테이너만 있는 웹 애플리케이션 서버
- **Apache**: 아파치 소프트웨어 재단에서 만든 Web Server로 정적 리소스를 응답 (80 포트)
- **Tomcat**: 동적인 데이터를 처리하는 서버. Web Application Server (8080 포트)



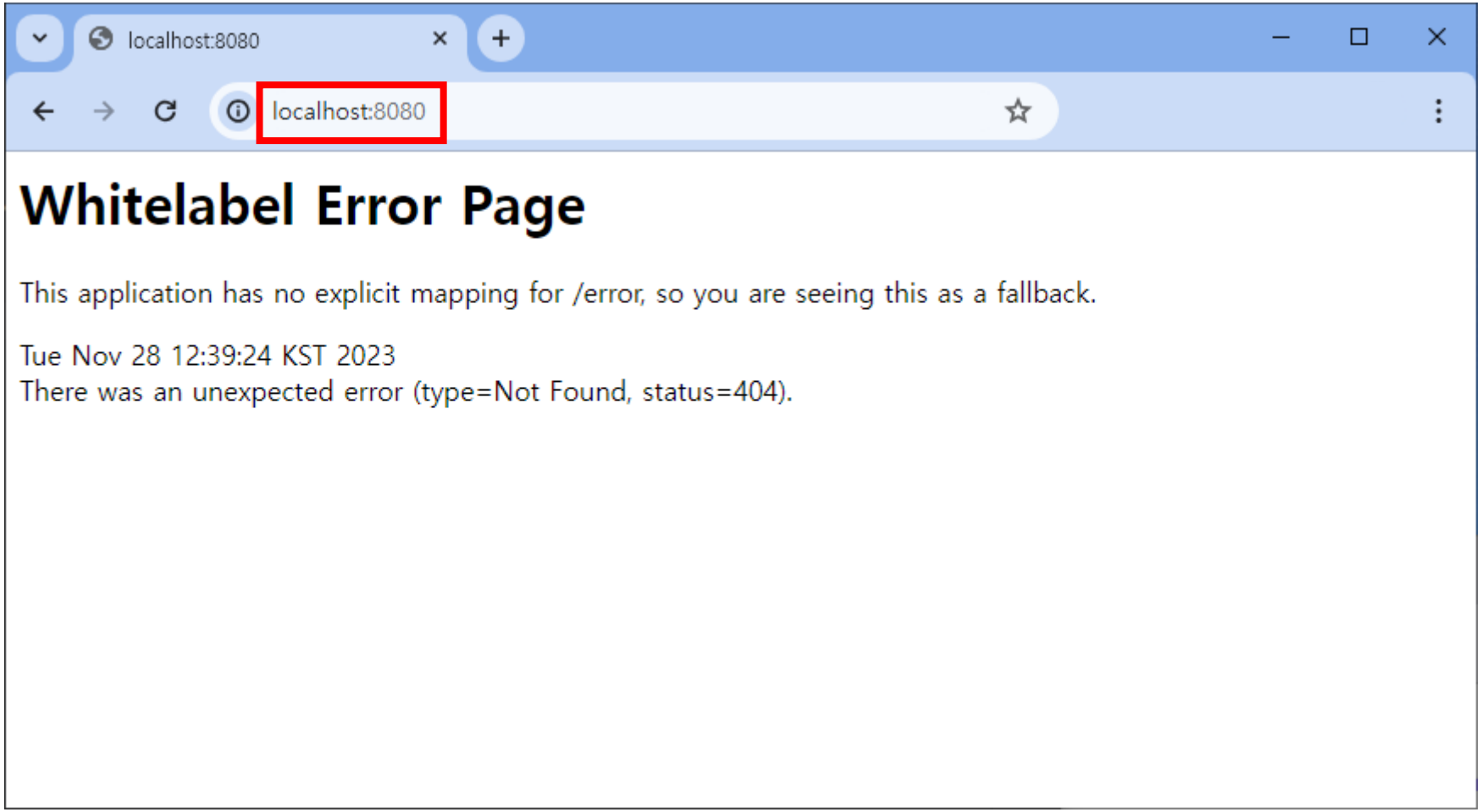
참고. 내장 AP 서버 (Embeded AP Server)

- **AP 서버** (애플리케이션 서버)
  - 자바 웹 애플리케이션을 구동하기 위해 필요한 프로그램 (ex. Tomcat, Jetty, Undertow 등)

- **내장 AP 서버**
  - 스프링 부트의 핵심 특징
  - 애플리케이션 내부에 AP 서버를 포함하므로 별도의 서버 설치가 필요 없음
  - 따라서, 애플리케이션 배포와 실행이 간단!

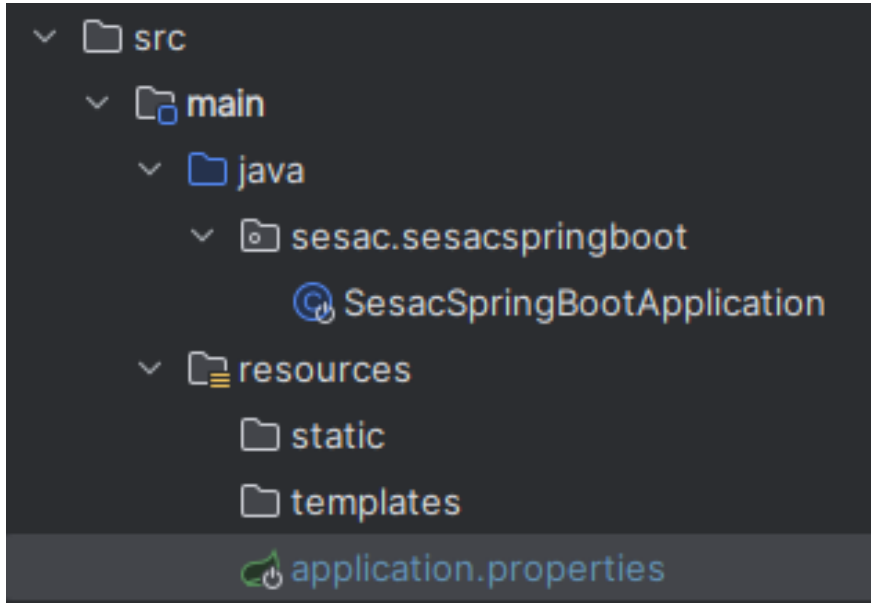


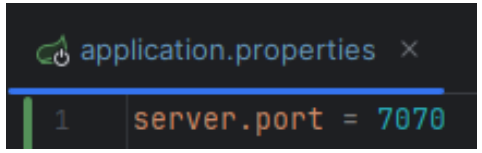
브라우저 접속하기


localhost:8080
localhost:8080
Whitelabel Error Page
This application has no explicit mapping for /error, so you are seeing this as a fallback.
Tue Nov 28 12:39:24 KST 2023
There was an unexpected error (type=Not Found, status=404).

참고. 서버 포트번호 변경하기

- 스프링 부트는 기본적으로 8080 포트번호를 사용해 내장 웹 서버를 실행
- src/main/resources 아래의 application.properties 파일에서 포트변호 변경 가능






```
Tomcat initialized with port 7070 (http)
Starting service [Tomcat]
Starting Servlet engine: [Apache Tomcat/10.1.16]
Initializing Spring embedded WebApplicationContext
Root WebApplicationContext: initialization completed in 821 ms
Tomcat started on port 7070 (http) with context path ''
Started SesacSpringBootApplication in 1.488 seconds (process running for 1.794)
```

참고. application.properties 파일

- Spring Boot 애플리케이션의 설정을 관리하는 중요한 파일

- 애플리케이션의 다양한 설정을 외부화하여 관리
- 환경별로 다른 설정을 쉽게 적용할 수 있음
  - application-{profile}.properties 형태로 환경 별 설정 가능 (ex. application-dev.properties)

- 보통 src/main/resources 디렉토리에 위치
- key=value 형태의 텍스트 기반 속성 파일

* Node 에서의 환경변수 파일(.env)과 역할이 유사함!

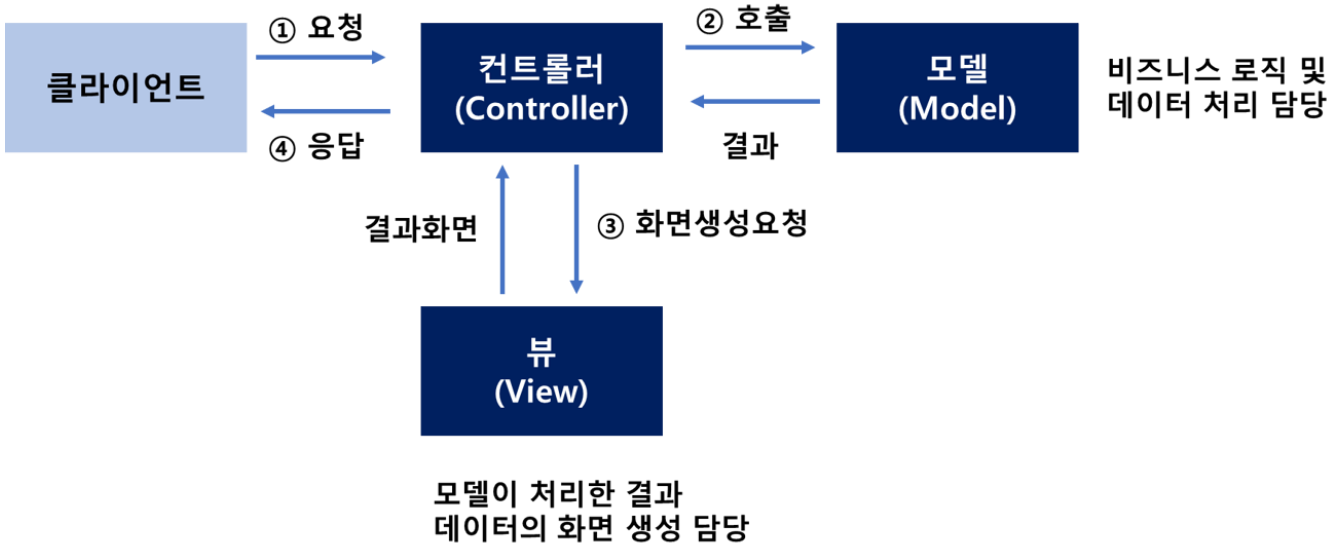
프로젝트 구조 살펴보기

- main/ : 실제 코드를 작성하는 곳
- test/: 프로젝트의 소스 코드를 테스트하는 코드 or 리소스 파일
- build.gradle: 빌드 설정 파일 * Node 에서의 package.json 파일과 역할이 유사함!
- setting.gradle: 빌드할 프로젝트 정보 설정 파일

Spring MVC

복습. MVC 패턴

- Model: 업무 로직, 업무 데이터 보유, 데이터베이스 접근 등
- View: HTML 생성
- Controller: 요청에서 응답까지의 흐름 제어



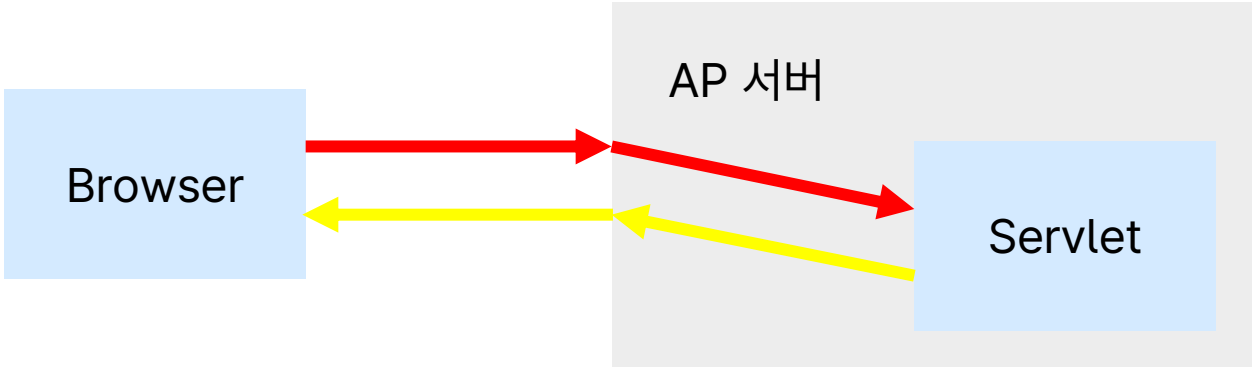
Spring MVC

- Spring 에서 제공하는 웹 모듈로 웹 애플리케이션을 빌드하기 위한 프레임워크
- Model, View, Controller 의 3가지 구성 요소
  - Model: 데이터와 비즈니스 로직 처리
  - View: 사용자에게 보여지는 부분
  - Controller: 사용자의 요청을 받아 처리하고 적절한 Model 을 호출한 후, 그 결과를 View 에게 전달

- MVC 패턴을 지원하고 DispatcherServlet 이라는 특수 서블릿을 통해 요청 처리
  - DispatcherServelt은 스프링 MVC의 핵심으로 모든 종류의 요청을 받아 적절한 Controller 에게 전달하고 그 결과를 다시 사용자에게 반환

참고. Servlet

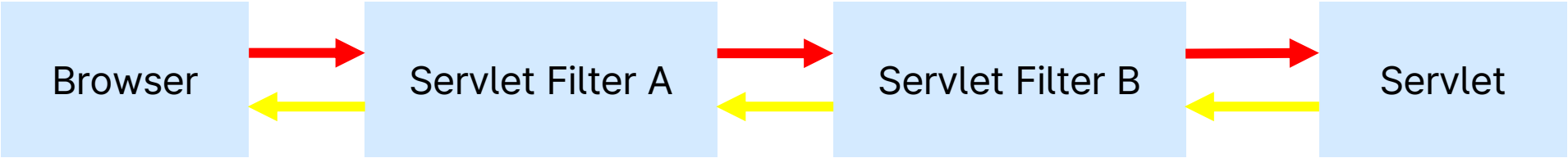
- 요청을 수신하고 응답을 반환하는 프로그램을 만들기 위한 자바 표준 기술
- 일반적으로 HTTP 프로토콜 사용 (프로토콜 종류에 제한이 있진 않음)
- Servlet 클래스를 AP 서버(애플리케이션 서버)에 등록하면 HTTP 요청에 대응하여 AP 서버가 Servlet 객체의 메서드를 호출



참고. Servlet Filter

- Servlet 처리 전후에 임의의 처리를 끼워 넣기 위한 자바 표준 기술
- Servlet Filter 여러 개를 연결 할 수도 있음

- ~~지금은 이해하기 어려우니.. 나중에 사용해보자!~~

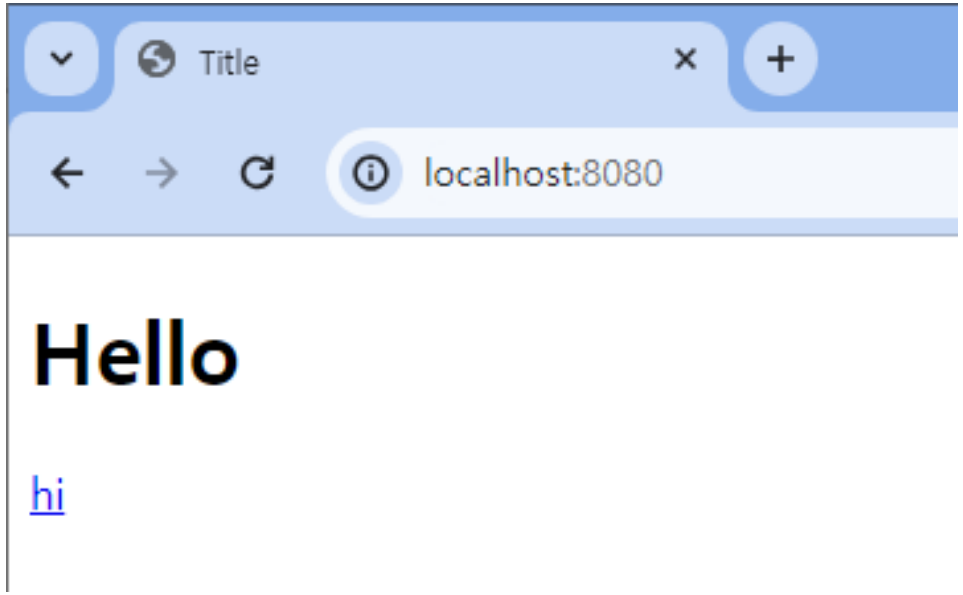


정적 파일

실행 화면 보여주기

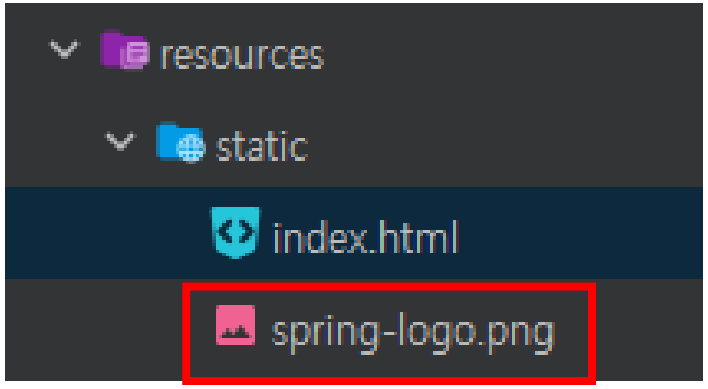
- src/main/resources/static에 index.html 생성

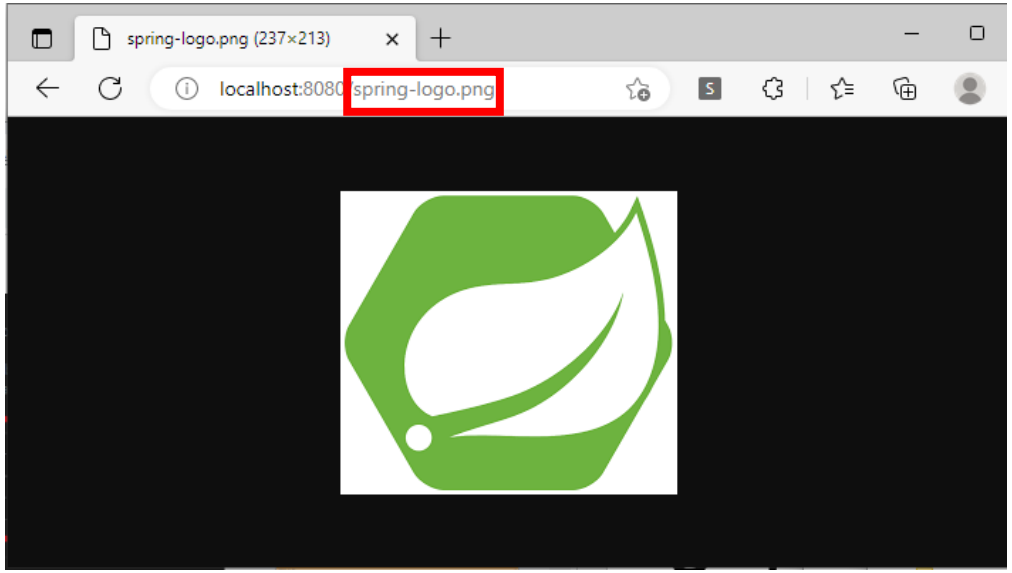
```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Title</title>
</head>
<body>
  <h1>Hello</h1>
  <a href="/hi">hi</a>
</body>
</html>
```



정적 파일

- src/main/resources/static 는 정적 파일 위치
- localhost:PORT 로부터 경로를 입력하면 위치하고 있는 파일 보기 가능





Thymeleaf 템플릿

Thymeleaf 란?

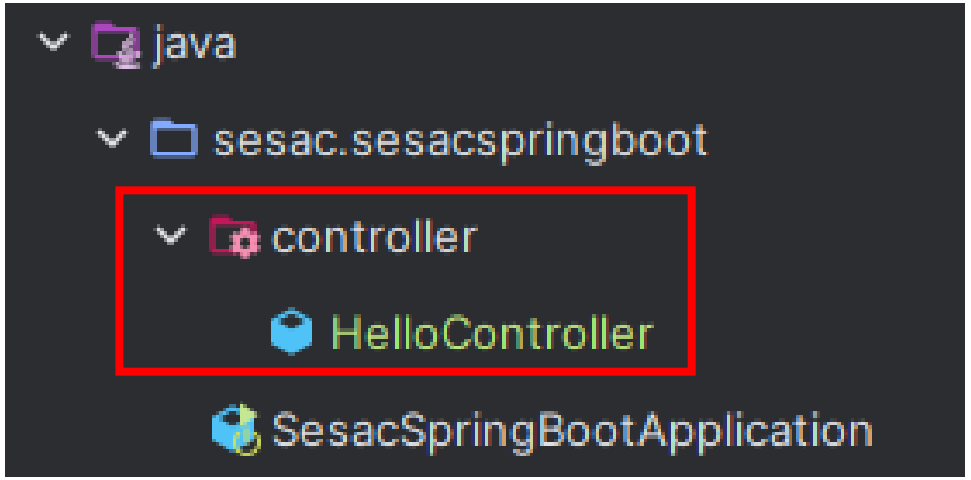
- 오픈 소스 템플릿 엔진
  - ex. JSP, Thymeleaf, FreeMarker, …
- HTML 태그에 속성을 추가해 페이지에 동적으로 값을 추가하거나 처리할 수 있게 도와주는 것
- Spring Boot 사용시 권장되는 템플릿 엔진

Thymeleaf

https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html

Controller 클래스 생성

- 임의의 클래스를 생성하고 클래스에 @Controller 추가
- Spring MVC에 Controller 클래스로 인식되어 스프링 MVC가 제공하는 다양한 어노테이션 사용 가능
- 해당 클래스에 요청에 대응하는 메서드를 작성



Controller 클래스 생성

```
package sesac.sesacspringboot.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class HelloController {

    @GetMapping("/hi")
    public String getHi(Model model) {
        model.addAttribute("msg", "Hi~");
        return "hi";
    }
}
```

@Controller

: 해당 클래스가 Controller 클래스라는 것을 Spring Container에게 알려준다.

@GetMapping

: URL을 매핑시켜 주는 것으로 HTTP의 GET 메서드의 /hi 경로에 대한 요청이 연결되며 getHi라는 함수를 실행시킨다.

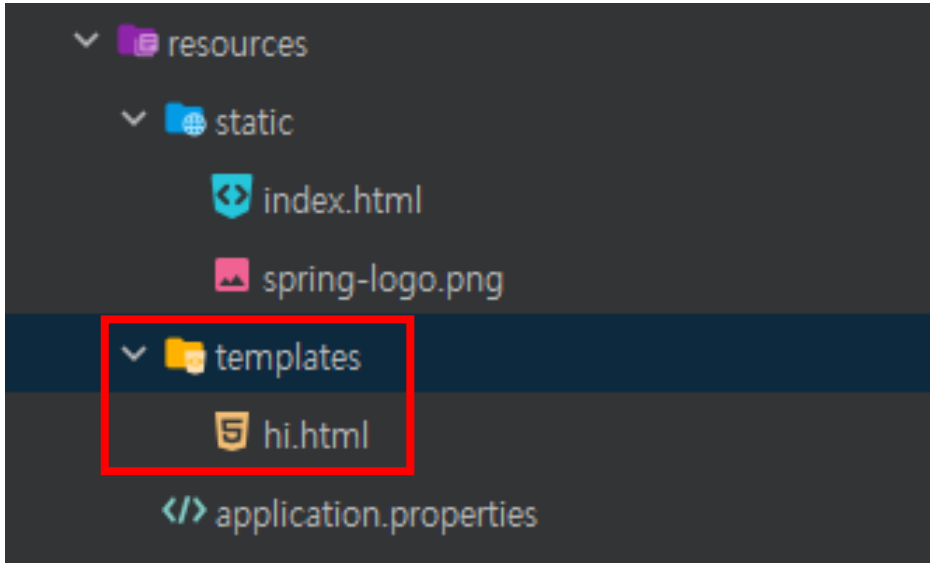
getHi() 메서드의 매개변수 타입 Model

: 스프링 MVC가 제공하는 타입으로 View에서 참조할 수 있는 객체 지정 가능

참고. **@XXX** 형식의 키워드들은 "**Annotation**" 단원에서 자세히 다룰 예정!

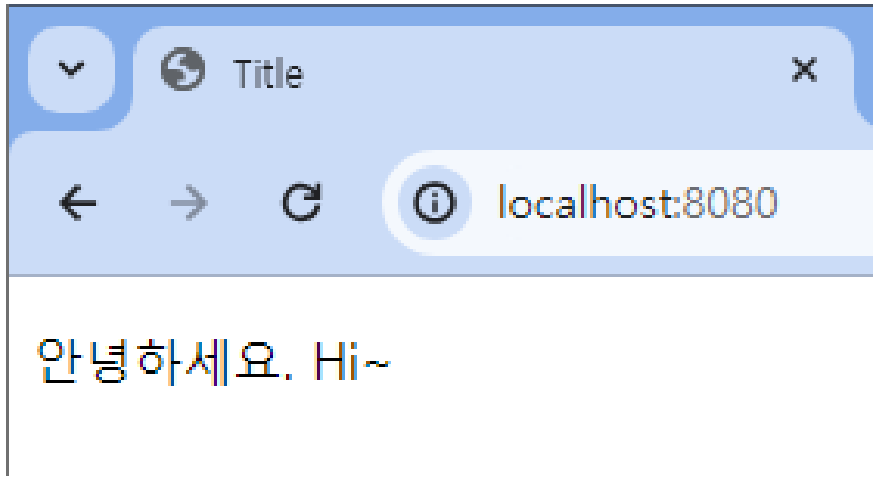
Template View 생성

- src/main/resources 아래 templates 폴더 아래 생성
- 템플릿 파일의 저장 위치와 확장자는 스프링 부트에 자동 설정되어 있음
  - 즉, view 파일은 templates/ 폴더에 생성되고, .html 확장자를 사용하도록
  - 여기서는 templates/hi.html 파일을 불러오는 것!

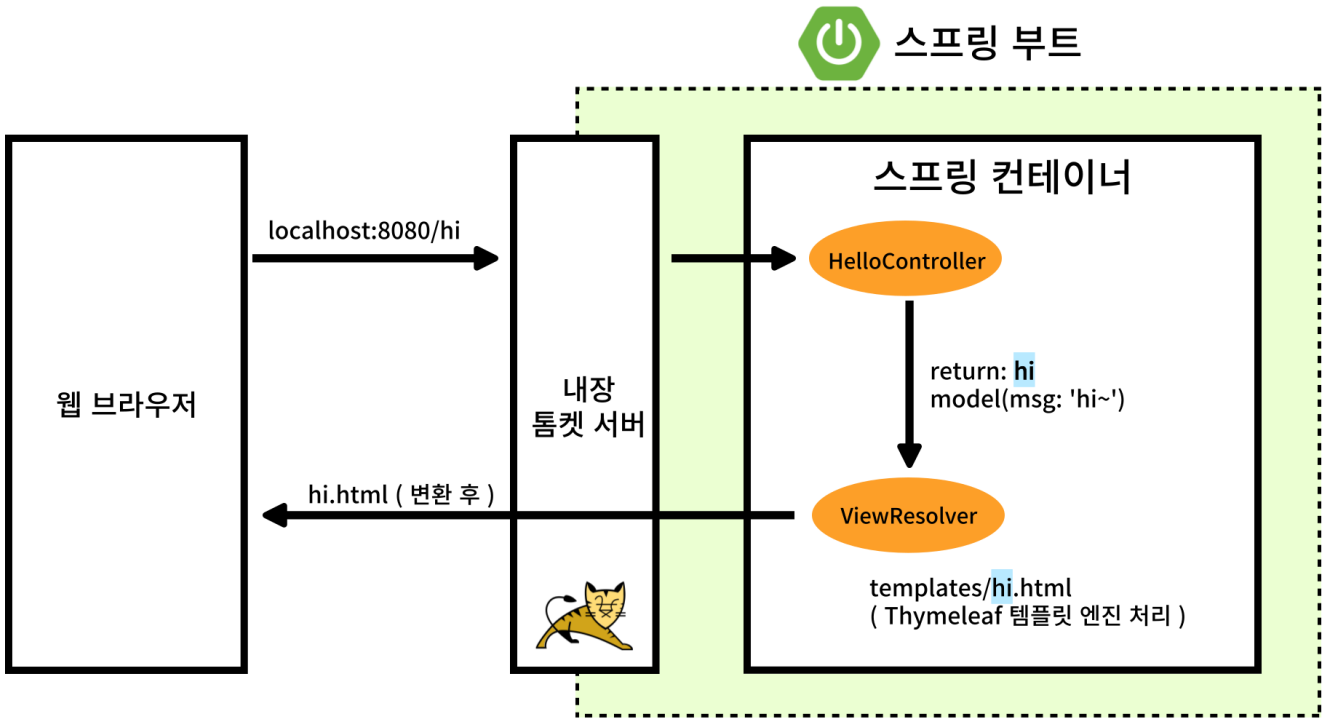


Template View 생성

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
   <meta charset="UTF-8">
   <title>Title</title>
</head>
<body>
   <p th:text="'안녕하세요. ' + ${msg}">안녕 반가워</p>
</body>
</html>
```



Spring Boot 동작 환경



스프링 부트는 WAS로 Tomcat을 내장

1. localhost:8080/hi 요청 시, 톰캣이 스프링한테 넘겨줌
2. 컨트롤러(HelloController.java)에서 매칭되는 URL 을 찾음
3. 해당 메소드(getHi) 에서 반환하는 문자열을 ViewResolver 가 화면을 찾아서 처리 (templates/hi.html)

Thymeleaf 표현식과 문법

Thymeleaf 표현식

- 서버에서 전달받은 데이터를 사용자들이 볼 수 있는 뷰로 만들기 위해 사용되는 표현식

표현식	설명
${…}	변수의 값 표현식
#{…}	속성 파일 값 표현식
@{…}	URL 표현식
*{…}	선택한 변수의 표현식 th:object 에서 선택한 객체에 접근

Thymeleaf 문법

- HTML 태그 안에 소 문법을 추가
- div 태그 뿐만 아니라 HTML 에서 지원하는 태그들 모두 사용 가능

```
<div th:[속성]="서버에서 받는 값 및 조건식" />
```

th:text

- 태그 안의 텍스트를 서버에서 전달받은 값에 따라 표현하고자 할 때 사용하는 문법

```
<span th:text="${hello}">message</span>
```

th:utext

- th:text와 유사
- 변수에서 받은 값에 html 태그가 있다면 태그 값을 반영해서 표시해줌
- “안녕” 값이 서버에서 넘어왔다면 이라는 글자가 나오는 것이 아닌 “안녕”이라는 글자가 태그에 의해 굵게 표시됨

th:value

- HTML Element 의 value 속성 값을 지정할 때 사용

```
<button th:value="${hello}" />
```

th:with

- 변수 값을 지정해서 사용하고자 할 때 사용

```
<div th:with="temp=${hello}" th:text="${temp}" />
```

- 위와 같이 사용한다면 서버에서 전달받은 'hello' 값이 저장된 temp 라는 변수가 만들어지고, Thymeleaf 문법 내에서 사용 가능

th:switch

- switch-case문을 이용할 때 사용되는 문법
- th:case 에서 case 문을 다루고, * 로 case문에서 다루지 않은 모든 경우가 처리됨
  (* 는 일반 switch-case 문법에서의 default)

```
<div th:switch="${hello}">
  <p th:case="'admin'">hello is admin</p>
  <p th:case="'manager'">hello is manager</p>
  <p th:case="*">hello is other</p>
</div>
```

th:if

- 조건문이 필요할 때 사용되는 문법
- Else 문이 필요한 경우에는 th:unless 를 사용

```
<p th:if="${hello}=='web'" th:text="${hello}"></p>
<p th:unless="${hello}=='web'" th:text="unless입니다."></p>
```

- 주의. th:unless 에 if에 적은 조건을 적어줘야 위의 if가 아닌 경우를 인식 가능!!

th:each

- 반복문이 필요한 경우에 사용되는 문법
- 리스트와 같은 collection 자료형을 서버에서 넘겨주면 그에 맞춰 반복적인 작업이 이루어질 때 사용 가능

```
String[] names = {"kim", "lee", "hong", "park", "shin"};
model.addAttribute( attributeName: "names", names);
return "05_Thymeleaf";
```

```
<ul>
  <li th:each="name:${names}">
    <span th:text="${name}">이름</span>
  </li>
</ul>
```

- kim
- lee
- hong
- park
- shin

실습. Thymeleaf (1)

1. Controller에서 age를 보냈을 때 20세보다 적으면 아래와 같이 출력

10세는 미성년자입니다.

2. Age가 20보다 크다면 아래와 같이 출력

20세는 성인입니다.

실습. Thymeleaf (2)

1. Controller에서 name과 age 정보를 갖고 있는 Person 클래스 만들기
2. ArrayList에 각기 다른 정보를 갖고 있는 Person 최소 4개 넣기
3. http://localhost:8080/people 에 접속했을 때 최소 4명의 사람에 대한 이름과 나이를 table 로 보여주기

이름	나이
kim	10
lee	20
hong	30
park	40
shin	50